JSP

R3.01

Nous avons vu

Les servlets HTTP un moyen de répondre à des requêtes HTTP mais dans la pratique nous avons eu dans la servlet coexistent la logique, la vue et les données.

De plus l'utilisation du « writer » de reponse est lourde :

```
// 1. Définir le type de contenu ET l'encodage AVANT d'appeler getWriter()
response.setContentType("text/html; charset=UTF-8");
// 2. Récupérer le writer
PrintWriter out = response.getWriter();
// 3. Écrire la réponse
out.println("<h1>Hello World</h1>");
// 1. Définir le type de contenu ET l'encodage AVANT d'appeler getWriter()
```

Jakarta Server Page

Un langage de script qui unie des balises et du code java (fragment)

```
<%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8" %>
<!DOCTYPE html>
<html lang="fr">
<head> <meta charset="UTF-8"> <title>Liste avec boucle JSP</title> </head>
<body>
   <h1>Ma liste de 3 éléments</h1>
   <ul>
      <% for (int i = 1; i <= 3; i++) { %>
         <li>Élément <%= i %></li>
      <% } %>
   </ul>
</body>
</html>
```

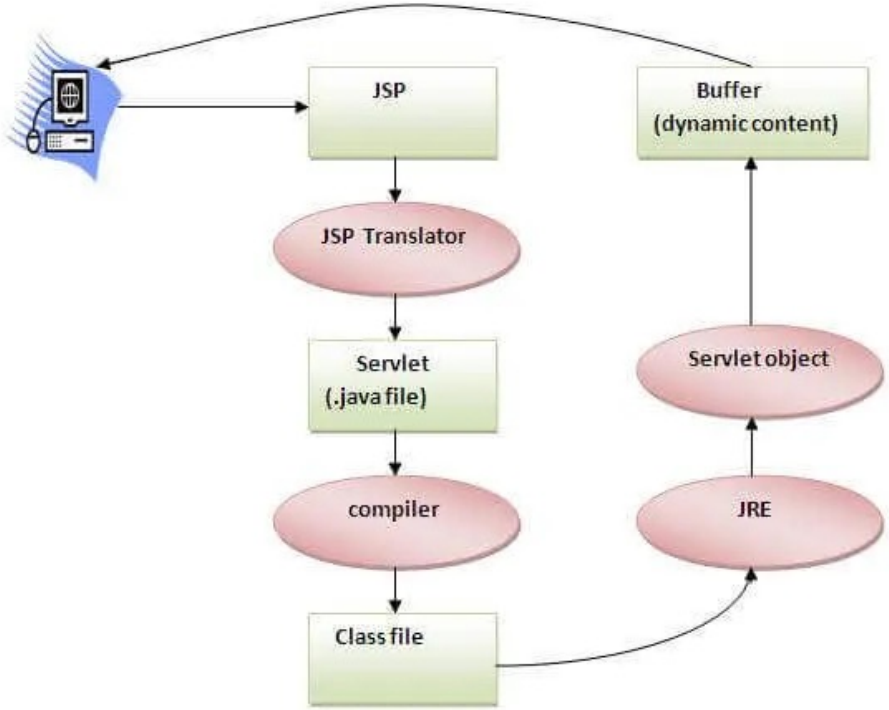
Site : https://www.jmdoudoux.fr/java/dej/chap-jsp.htm

JSP cycle de vie

Une JSP sera transformée en servlet :

- traduction (transpilation)
  - A la première consultation de la page, le conteneur analyse le fichier .jsp et le transforme en servlet mon.jsp devient mon_jsp.java
- compilation
  - Le compilateur Java transforme le fichier .java en bytecode (.class).
- chargement et l'instanciation
  - Le conteneur charge la classe .class dans la mémoire Java (JVM) et instancie la servlet
- initialisation
  - La méthode jspInit() est appelée une seule fois par le conteneur après l'instanciation de l'objet.
- traitement des requêtes : _jspService(request, response)
  - méthode appelée à chaque requête
- destruction
  - à l'arrêt de l'application web ou lorsque que le conteneur décide de décharger la JSP, la méthode jspDestroy() est appelée.

Rem: il faudra debugger coté serveur

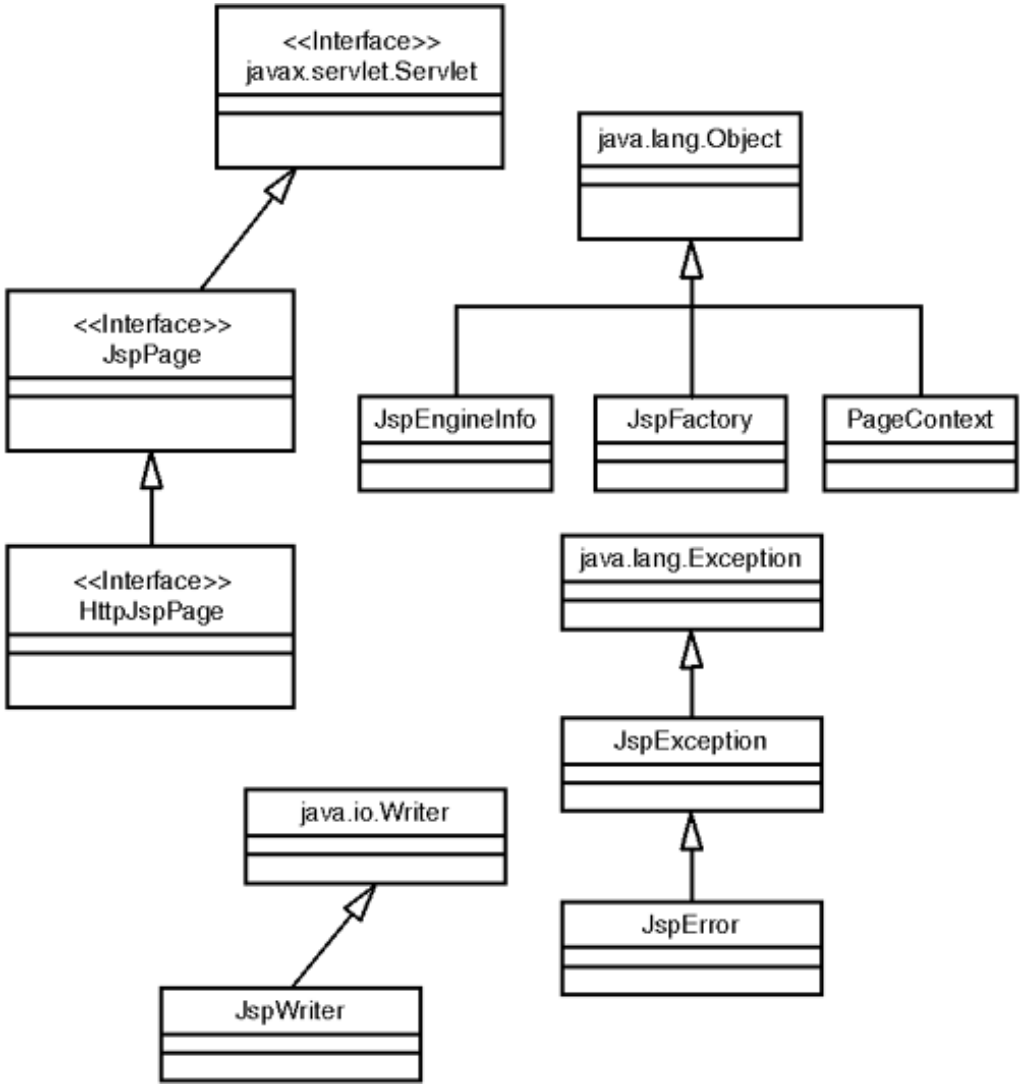


JSP principe

Les pages JSP reposent sur :

- les données statiques comme le HTML,
- les directives,
- les scripts et variables,
- les actions,
- les balises personnalisées

Les classes



Pure Java Server Pages
(James Goodwill)

Classe

La servlet générée :

- Une servlet qui implémente l'interface `HttpJspPage`
  - Une interface qui hérite de `JspPage` qui elle même hérite de `Servlet`
  - Son point d'entrée est la méthode `_jspService`

Interface HttpJspPage

- **Méthode spécifique** : Elle ajoute explicitement la déclaration de la méthode _jspService(HttpServletRequest request, HttpServletResponse response) dans le système de types Java
- **Rôle** : Lorsqu'un fichier .jsp est compilé par le conteneur Web (comme Apache Tomcat), la classe générée implémente HttpJspPage pour traiter les requêtes et réponses HTTP via _jspService().
- **Cycle de vie** : Elle hérite des méthodes jspInit() et jspDestroy() de l'interface JspPage

Script JSP

- Commentaires (ne sont pas produits en sortie) <%-- expression java --%>
- Directives : <%@ ... %>
- Déclarations (attributs et méthodes) : <%! Déclarations de variables et de méthodes %>
- Scriplets : <% bloc d’instructions %>
- Expressions <%= expression java %>

Déclaration <%! %>

```
<%!
        //variable d’instance
        private String nomApplication;
        private long heureDemarrage;
        // Redéfinition de la méthode jspInit()
        @Override public void jspInit() {
        nomApplication = "Mon Application Web E-Commerce";
        heureDemarrage = System.currentTimeMillis();
        System.out.println("La JSP a été initialisée avec succès !"); }
        // Méthode utilitaire public long ObtenirTempsActivite() {
        return (System.currentTimeMillis() - heureDemarrage) / 1000; }
        // Redéfinition optionnelle de jspDestroy() lorsque la JSP est déchargée
        @Override public void jspDestroy() {
        System.out.println("La JSP est en train d'être détruite. Libération des ressources."); } %>
```

Rem: exécuté une seule fois

Scriptlet <% %>

```
<% // Ce code s'exécute à chaque consultation
i++;
If (…) {
}
%>
```

Scriptlet <% %>

Un scriptlet peut être fragmenté => mélanger HTML et code

```
<% for (int i = 1; i <= 5; i++) { %>
 <!-- Ce code HTML est répété par la boucle Java -->
        <li>Ligne numéro <%= i %></li>
<% } // Fermeture de la boucle for %>
```

Expression <%= %>

Elle est utilisée pour évaluer une unique expression Java, la convertir automatiquement en chaîne de caractères (String), et **l'insérer directement dans le flux HTML** envoyé au navigateur du client.

<!-- 1. Afficher des données du système ou des calculs directs -->

Date et heure du serveur : **<%= new java.util.Date() %>** Calcul mathématique (5 * 5) : **<%= 5 * 5 %>**

Chaîne en majuscules : **<%= "java ee".toUpperCase() %>**

<!-- 2. Afficher des variables locales préalablement créées -->

<% String nomClient = "Jean Dupont";

double solde = 1500.75; %>

Client : **<%= nomClient %>**

Solde de votre compte : **<%= solde %> €**

<!-- 3. Récupérer et afficher un paramètre de l'URL (ex: ?recherche=jsp) --> Votre recherche : **<%= request.getParameter("recherche") %>**

…

Directive

Syntaxe: <%@ directive attribut="valeur" %>

Trois directives : page, include, Taglib :

page <%@ page import="..." %>

Gère les imports de packages Java, l'encodage, les pages d'erreur ou la gestion des sessions.

include <%@ include file="..." %>

Fusionne le contenu d'un autre fichier (HTML ou JSP) au moment de la compilation de la page.

taglib <%@ taglib prefix="..." %>

Déclare l'utilisation d'une bibliothèque de balises personnalisées (comme JSTL pour éviter le code Java).

Directive Page

<%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8" %>

<%@ page import="java.util.ArrayList, java.time.LocalDate" %>

<%@ page errorPage="erreur.jsp" %>

Directive Include

<%@ include file="header.jsp" %> … <%@ include file="components/footer.html" %>

Caractéristique	Directive <%@ include file="..." %> (Statique)	Action <jsp:include page="..." /> (Dynamique)
Moment de l'inclusion	À la compilation (Traduction de la JSP en Servlet)	À l'exécution (Chaque fois que la page est demandée)
Fonctionnement	Le code du fichier cible est "copié-collé" à l'intérieur de la JSP maîtresse avant de compiler le tout en un seul bloc.	La page principale s'exécute, appelle la page secondaire, récupère son flux HTML et l'insère.
Performance	Plus rapide car le traitement est fait une seule fois.	Légèrement plus lent car il y a deux requêtes internes au serveur.
Variables	Partage les variables Java locales entre les fichiers.	Ne partage pas les variables locales (nécessite de passer par request).

Directive taglib

La directive **taglib** s'écrit avec la syntaxe **<%@ taglib uri="..." prefix="..." %>**.

Elle permet d'importer des bibliothèques de balises personnalisées (Custom Tag Libraries) dans une page JSP.

<%-- 1. Déclaration de la bibliothèque JSTL Core avec le préfixe 'c' --%>
<%@ taglib uri="http://sun.com" prefix="c" %>

Objet accessible nativement au sein d'une JSP

Nom de variable	Type java
request	javax.servlet.http.HttpServletRequest
response	javax.servlet.http.HttpServletResponse
session	javax.servlet.http.HttpSession
application	javax.servlet.ServletContext
out	javax.servlet.jsp.JspWriter
pageContext	javax.servlet.jsp.PageContext
config	javax.servlet.ServletConfig
page	java.lang.Object (HttpJspPage)
exception	java.lang.Trowable

Context en JSP

Niveau de Contexte	Portée / Durée de vie	Objet Java associé	Usage typique
page	Limité à la page JSP actuelle.	pageContext	Variables temporaires de calcul.
request	Valide pour une seule requête HTTP (survit aux redirections internes forward).	request	Transmettre des données d'une Servlet à une JSP.
session	Propre à un client HTTP	session	Partager de l'information entre les pages pour une même session.
application	Globale à l'application	application	Compteur global de visites, configuration de l'application.

Exemple 1 : Un tableau de produits

MVC :

- Un DAO qui gère des Product
- Une servlet qui gère les actions
- Une JSP qui affiche

Exemple 1 : Un tableau de produits JSP

Les produits sont obtenus de la servlet

```
<% List<Product> products =
(List<Product>) request.getAttribute("products");%>
```

Les produits sont affichés

```
<% for (Product product : products) { %>
<tr>
  <td><%= product.getId() %></td>
…
```

Des actions sont possibles (ajouter, supprimer une quantité)

```
form method="post"
      action="${pageContext.request.contextPath}/products/products_controller">
  <input type="hidden" name="id"
      value="<%= product.getId() %>">
  <input type="hidden" name="action" value="increment">
  <button type="submit">+</button>
</form> </td>
```

Exemple 1 : Un tableau de produits Servlet

Consulte le DAO

```
ProductMemoryDao productDao = (ProductMemoryDao) dao;
Optional<Product> product = productDao.get(id);
```

Analyse le traitement

```
if ("increment".equals(action)) {
…
```

Forward ou redirige

```
request.getRequestDispatcher("/JSP/Products/products_view.jsp")
        .forward(request, response);
```

Exemple 2

- Un formulaire de login
  - url demo/JSP/sessions/ session-login.jsp
  - Champs : login, password

- Une servlet qui gère l'authentification, la création du cookie, la création de la session
  - url : /demo/jsp/session-authenticate

- Une page JSP accessible uniquement si la session existe et affiche le login présent dans la session
- url demo/JSP/sessions/ session-login.jsp

Exemple 2 : formulaire de login

```
<%@ page contentType="text/html;charset=UTF-8" language="java"
%>
<%@ page import="jakarta.servlet.http.Cookie" %>
<%@ page import="java.util.Arrays" %>


<%
   Cookie[] cookies = request.getCookies();
%>


value="<%= loginCookie != null ? loginCookie : "" %>"
```

Exemple 2 : la servlet

```
//recupère les paramètres
String login = request.getParameter("login");
String password = request.getParameter("password");

//redriection
response.sendRedirect(referer);

//Crée le cookie response.addCookie(cookie);
Cookie cookie = new Cookie("login", login);
//l’ajoute à la réponse
response.addCookie(cookie);

//Créer la session
HttpSession session = request.getSession();
session.setAttribute("login", login);
```

Exemple 2 : la page accessible si session

```
//recupération du login de la session
HttpSession sessionUser =
        request.getSession(false);
sessionUser.getAttribute("login")
//redirection si echec
response.sendRedirect("session-login.jsp");

//recupération du login
String login = (String) sessionUser.getAttribute("login");
```

Les actions

Les **actions JSP** sont des balises XML prédéfinies qui permettent de contrôler le comportement du moteur JSP ou d'interagir avec des composants Java (comme les JavaBeans) **sans écrire de code Java**.

Syntaxe : Eléments XML "actifs" au sein d'une page JSP

<prefixe:action attr="…"> … </prefixe:action>

Effet : Exécuter du code à chaque passage

Deux sortes d'actions :

Actions standards (prefixe jsp) exemple <jsp:action>

Actions définies par l'utilisateur : JSP tags (autres prefixes)

Les actions sont regroupées par librairies

Source : https://www.irisa.fr/LIS/Members/bekkers/javaj2ee/view.html

Actions Standard

- Modularité
  - `<jsp:include>` inclure un fichier en place
  - `<jsp:forward>` transférer le contrôle à une autre page
- Utiliser les beans
  - `<jsp:useBean>` trouver (instancier) un bean
  - `<jsp:getProperty>` valeur d'un attribut de bean
  - `<jsp:setProperty>` initialise un attribut de bean
- Appel d'une applet
  - `<jsp:plugin>` génére du code `<OBJET>` ou `<EMBED>`

Servir une requète à l'aide de plusieurs pages jsp

- A quoi cela sert
  - Récupérer les erreurs
  - Simplifier le contrôle
  - ou simplement séparer contrôle et traitements
  - Modulariser les programmes
- Quels outils sont nécessaires
  - Des outils pour passer le contrôle d'une page à une autre
  - Des outils pour passer de l'information d'une page à l'autre

<jsp:forward>

- Action <jsp:forward>

```
<jsp:forward page="page1.jsp"/>
```

- Attribut page
  - Reférence relative à la racine de l'application

```
page="/rep/page1.jsp"
```

  - Reférence relative à la page d'appel

```
page="rep/page1.jsp"
```

- Effet : arrête l'exécution de la page courante, lance l'exécution d'une autre page

<jsp:forward> suite

- Effets (suite)
  - L'URI d'appel reste inchangée (en cas de rechargement dans le navigateur, c'est la page d'appel qui est rechargée …)
  - Les variables request, result … restent accessibles
  - Le buffer de sortie est réinitialisé (si le transfert avait déjà commencé, une erreur est générée)
- Passer de l'information entre pages :

```
<jsp:forward page="myPage.jsp">
    <jsp:param name="nom" value="val"/>
</jsp:forward>
```

Récupération d'erreurs

```
<% try {
…
   } catch(Exception e) {
   %>
   <jsp:forward page="erreur.jsp">
    <jsp:param name="titre" value="Erreur
pendant la lecture de la base"/>
    <jsp:param name="message"
         value="<%= e.getMessage()%>"/>
 </jsp:forward>
<%}%>
```

<jsp:include>

- Inclure en ligne la réponse d'une autre page jsp, servlet ou page statique
  - L'URI reste celle de la page d'appel
  - Les informations sur la requète restent inchangées
  - Le buffer de sortie est vidé sur la sortie avant de commencer
  - Des paramètres peuvent être passés

```
<jsp:include page="nav.jsp">
   <jsp:param name="nom" value="val"/>
</jsp:include>
```

<jsp:plugin>

- Génération d'un élément <embed> ou <object> (en fonction du navigateur utilisé)

```
<jsp:plugin type="applet"
    code="Clok.class" jreversion="1.2"
    width="160 value="150">
    <jsp:params>
        <jsp:param name="nom" value="val"/>
    </jsp:param>
</jsp:plugin>
```

Actions standards pour utiliser les java beans

Les Beans - conventions

- bean = classe qui respecte des conventions

```
public class MonBean {
   // Attributs privés
      private type XXX;
   // constructeur
      public MonBean() {…}
     …
    // Lecture/écriture d'un attribut
   type getXXX() { … }
   setXXX(type val) { … }
}
```

Attribut caché

Constructeur (sans paramètre)

Visibilité gérée par des méthodes d'accès

Actions jsp standards – les "beans"

- Trois actions standards pour les beans

```
<jsp:useBean id="XXX" class="…"/>
<jsp:getProperty name="XXX" property="…"/>
<jsp:setProperty name="XXX" property="…"
   value="…"/>
```

- Trois éléments vides (en général) :
  - usage des paramètres pour passer l'information
  - Attention *XXX* doit être un identificateur java valide
- Trois traitements correspondants
  - *Déclaration/création d'une instance d'objet*
    (constructeur sans paramètre)
  - *Lecture d'un attribut de classe*
  - *Écriture d'un attribut de classe*

Création d'un bean

- Création d'un bean dans une page jsp

```
<jsp:useBean id="horloge"
        class="myPackage.DateFormatee"/>
```

- Code Généré : deux fonctions
  - Création d'une instance d'objet

```
myPackage.DateFormatee horloge = null;
horloge = new myPackage.DateFormatee();
```

  - Réutilisation du bean : le bean est sauvé par défaut dans le contexte de la requête

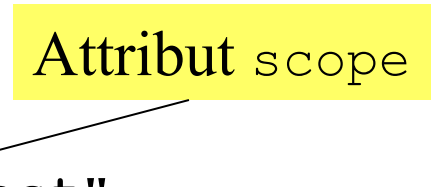
Contexte d'un bean

- Quatre contextes d'exécution imbriqués (rappel)

  Page<requète<session<application

- Action `<jsp:useBean>`

```
<jsp:useBean id="ident" scope="request"
    class="lib.Classe"/>
```



- Si l'objet bean existe déjà dans la portée on utilise celui-ci et on ne crée pas de nouvel objet

Création d'un bean attribut `@scope`

- Création par défaut au sein de la requête

```
<jsp:useBean id="horloge"
        class="myPackage.DateFormatee"/>
```

- Création dans le contexte de la session

```
<jsp:useBean id="horloge"
        class="myPackage.DateFormatee"
        scope="session"/>
```

Portée (attribut `scope`)

- `scope="page"`
  - objet est visible dans la page (valeur par défaut)
- `scope="request"`
  - objet est visible au fils des jsp:forward
- `scope="session"`
  - objet est visible pour la session
- `scope="application"`
  - objet est visible pour toute l'application

Attribut `@class` ou `@type`

- Au moins un attribut `class` ou `type` doit être présent
  - `class="`*`nom`*`"` : Un nom complet de classe
  - `type="`*`nom`*`"` : Un nom complet de super classe de l'objet ou d'interface implémentée

Elément <jsp:useBean> avec contenu

```
<jsp:useBean id="myBean" class="myClass">
   <jsp:setProperty name="myBean"
       property="prop" value="val"/>
</jsp:useBean>
```

Le corps de l'élément est exécuté une seule fois lors de la création de l'instance de l'objet bean

<jsp:useBean> avec contenu

```
<%@ page language="java" %>
<html><body>
  <h1>Utilisation du Bean
      <code>String</code></h1>

  <jsp:useBean id="monTexte" class="String">
     <p>On crée une chaîne nommée monTexte</p>
     <%monTexte = "blabla";%>
  </jsp:useBean>

  <p>monTexte =  <%= monTexte%></p>
</body></html>
```

<jsp:useBean> avec contenu - bis

```
<%@ page language="java" %>
<html><body>
  <h1>Utilisation du Bean <code>String</code></h1>

  <jsp:useBean id="monTexte" class="String">
     <p>On crée une chaîne nommée monTexte</p>
     <%monTexte = "blabla";%>
  </jsp:useBean>

  <p>monTexte =  <%= mon
</body></html>
```



Exemple de Bean -"*date courante formatée*"

```
package myPackage;
public class DateFormatee {
   static SimpleDateFormat sdf =
     new SimpleDateFormat(
      "EEEEEEEEE dd MMMMMMMMMMMM yyyy");
   private Date dateCreation;
   public DateFormatee () {
     dateCreation = new Date();
   }
   static public String getDate() {
     return sdf.format(new Date());
   }
   public String getDateCreation() {
     return sdf.format(dateCreation);
   }
}
```

Attention
« default package » interdit

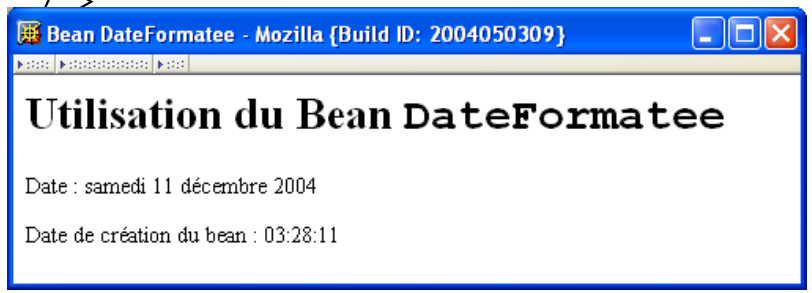
Attention les attributs **static** ne sont pas des propriétés du bean

Une seule propriété

Utilisation du bean

```
<%@ page language="java" %>
<html><body>
  <h1>Utilisation du Bean DateFormatee</h1>
  <p>Date :
    <%= myPackage.DateFormatee.getDate()%>
  </p>
  <jsp:useBean id="horloge"
    class="myPackage.DateFormatee"/>
  <p>Heure de création :
    <jsp:getProperty name="horloge"
      property="dateCreation"/>
  </p>
</body></html>
```

Attention les attributs **`static`** ne sont des propriétés du bean



Code généré pour l'accès à une propriété d'un bean

- Code JSP

```
<jsp:getProperty name="horloge"
        property="dateCreation"/>
```

- Code Java

```
(
  (myPackage.DateFormatee)
  _jspx_page_context.findAttribute(
    "horloge")
).getDateCreation()
```

Elément <jsp:setProperty>

Trois formes d'appel

- Valeur directe

```
<jsp:setProperty name="XXX"
   property="…" value="chaîne"/>
```

- Valeur résultant du calcul

```
<jsp:setProperty name="XXX"
   property="…" value="<%= expression %>"/>
```

- Valeur provenant d'un paramètre saisi dans un formulaire

```
<jsp:setProperty name="XXX"
   property="prop" Param="param"/>
```

(*prop*:=*param*)

Elément <jsp:setProperty>

```
<jsp:setProperty name="XXX"
   property="…" value="…"/>
```

- Le type d'une propriété est quelconque
- Si la valeur est une chaîne une conversion implicite est opérée

Conversions implicites de chaîne vers un type primitif de Java

Type de propriété	Méthode de convertion
`Boolean ou boolean`	`Boolean.valueOf(String)`
`Byte ou byte`	`Byte.valueOf(String)`
`Character ou char`	`String.charAt(int)`
`Double ou double`	`Double.valueOf(String)`
`Integer ou int`	`Integer.valueOf(String)`
`Float ou float`	`Float.ValueOf(String)`
`Long ou long`	`Long.valueOf(String)`

Initialisation d'un "bean"

- *Condition* : chaque paramêtre récupéré dans le formulaire possède un attribut de même nom dans la classe bean

```
<jsp:useBean id="monObjet"
    class="lib.MaClasse">
   <jsp:setProperty name="monObjet"
      property="*"
   />
</jsp:useBean>
```

Utilisation de *

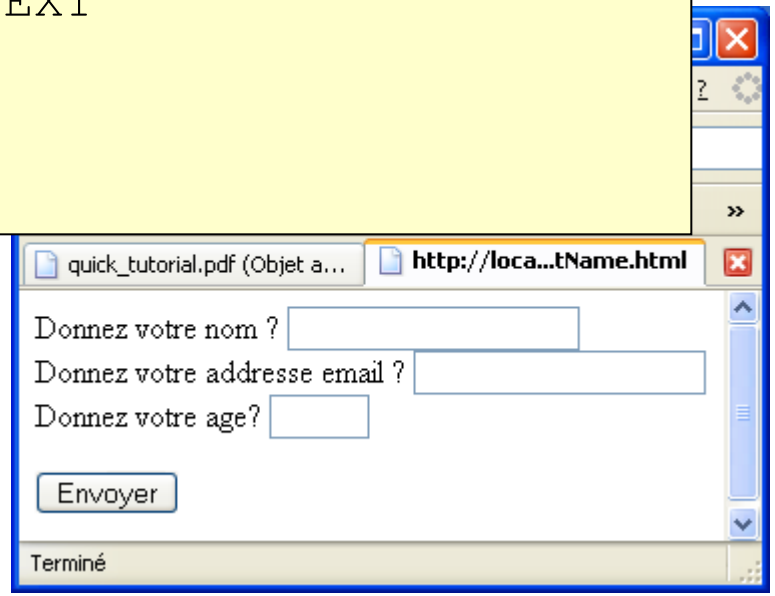
Usage des Java beans et des formulaires

exemple

Formulaire de saisie

getName.html

```
<HTML>
<BODY>
<FORM METHOD=POST ACTION="saveName.jsp">
Donnez votre nom ? <INPUT TYPE=TEXT
      NAME="username" SIZE=20><BR>
Donnez votre addresse email ? <INPUT TYPE=TEXT
      NAME="email" SIZE=20><BR>
Donnez votre age? <INPUT TYPE=TEXT
      NAME="age" SIZE=4>
<P><INPUT TYPE=SUBMIT>
</FORM>
</BODY>
</HTML>
```



Java beans (*cf* nom des attributs)

```
public class UserData {
   String username;
   String email;
   int age;

   public void setUsername(String value) {
            username = value;
   }
   public void setEmail(String value) {
            email = value;
   }
   public void setAge(int value) {
            age = value;
   }
```

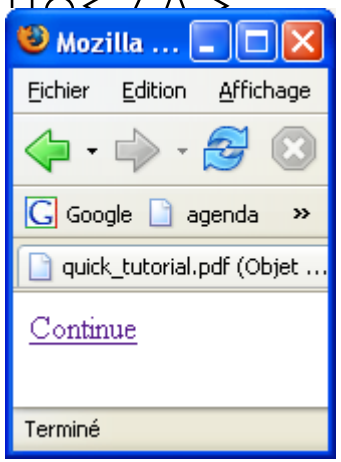
```
public String getUsername() {
  return username;
}
public String getEmail() {
  return email;
}
public int getAge() {
  return age;
}
}
```

Les getters et setters doivent être public, le nom des attribut est le même que dans le formulaire, la classe ne doit pas être dans le paquetage par défaut

Récolter les valeurs, page JSP

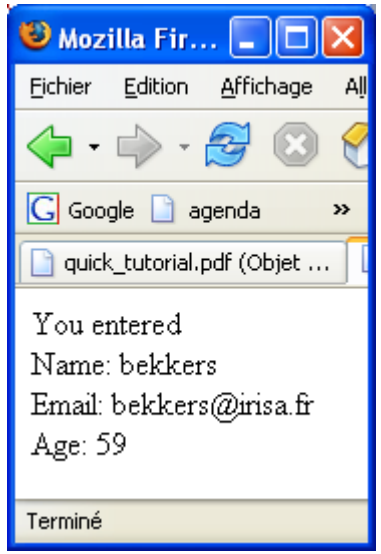
saveName.jsp

```
<jsp:useBean id="user"
  class="test.UserData" scope="session"/>
<jsp:setProperty name="user"
  property="*"/>
<HTML>
<BODY>
<A HREF="nextPage.jsp">Continue</A>
</BODY>
</HTML>
```



Afficher les valeurs
nextPage.jsp

```
<jsp:useBean id="user" class="test.UserData"
  scope="session"/>
<HTML>
<BODY>
You entered<BR>
Name: <%= user.getUsername() %><BR>
Email: <%= user.getEmail() %><BR>
Age: <%= user.getAge() %><BR>
</BODY>
</HTML>
```



5 librairies standard d'actions

- Classées en 5 familles
  1. Core (préfixe `c`)
  2. XML (préfixe `x`)
  3. Internationalisation i18n (préfixe `fmt`)
  4. Base de données (préfixe `sql`)
  5. Fonction diverses (préfixe `fn`)
     - sur les chaînes …

Librairies standards

- Définie par qui ?
  - Sun
- Disponibles où ?
  - Sur le site d’apache on en trouve une implémentation

URIs pour les librairies

1. *Core*: `http://java.sun.com/jsp/jstl/core`
2. *XML*: `http://java.sun.com/jsp/jstl/xml`
3. *Internationalisation*: `http://java.sun.com/jsp/jstl/fmt`
4. *SQL*: `http://java.sun.com/jsp/jstl/sql`
5. *Fonctions*: `http://java.sun.com/jsp/jstl/functions`

- Exemple de déclaration

```
<%@ taglib
      uri="http://java.sun.com/jstl/core"
      prefix="c" %>
```

Attention

- Les librairies de tag standard ne viennent pas par défaut :
  - il faut les charger sur le site d’Apache !

Éléments actifs « core »

Gestion de variables	`remove, set`
Gestion du contrôle	`choose (when, otherwise),` `forEach , forTokens, if`

Gestion des urls	`import (param),`
	`redirect (param),`
	`url (param)`

Divers	`catch, out`

Exemple gestion des variables

- Initialiser

```
<c:set var="bookId"
    value="${param.Remove}"/>
```

- Effacer

```
<c:remove var="cart"
    scope="session"/>
```

- Référencer

```
${bookId}
```

Affichage d'une liste

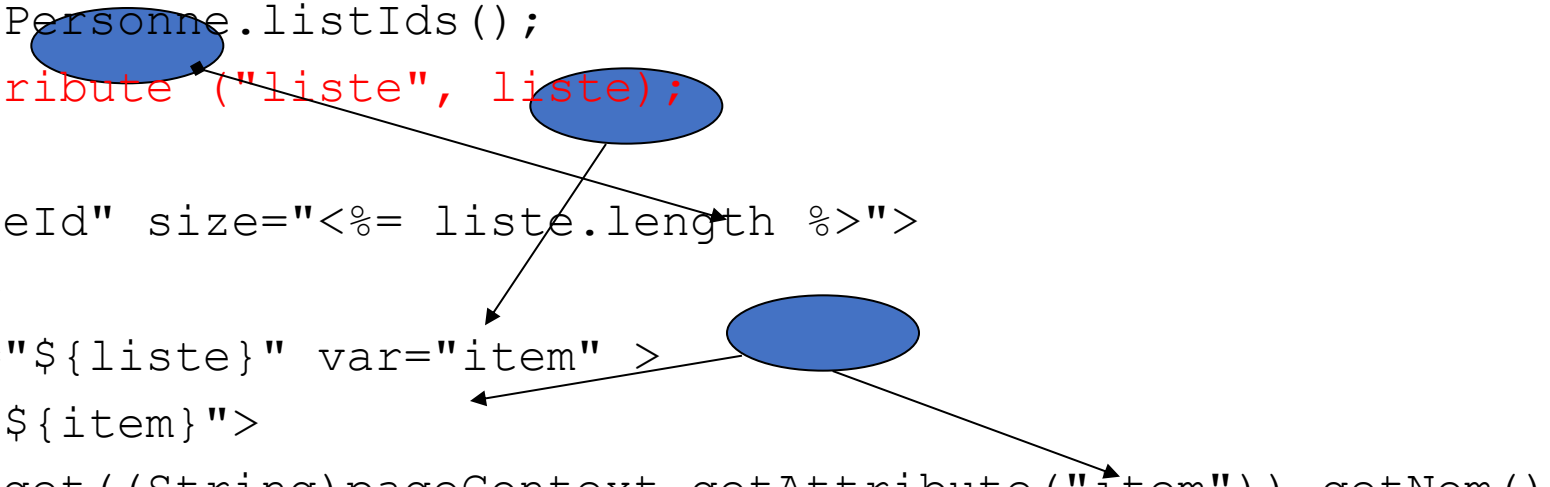
```
<ul>
<%   List bookList = (List) request.getAttribute ("books-list");
   if (bookList!=null) {
      Iterator iterator = bookList.iterator();
      int i = 0;
      while ( iterator.hasNext() ) {
         Book b = (Book) iterator.next();
         out.print ("<li class='"+(i%2==0?"pair":"impair")+"'>");
         out.print (b.getName()+" ("+b.getPrice()+" &euro;)");
         if ( b.getPrice() < 30.0 )
            out.print (" <img src='hot.png'
             alt='Moins de 30 &euro;'/>");
         out.println ("</li>");
         i++;
      }
   }
%>
</ul>
```

Utiliser la librairie `core`

```
<ul>
<c:forEach items="${requestScope['books-list']}"
  var="book" varStatus="status">
  <li class="${status.index%2==0?'pair':'impair'}">
        ${book.name} (${book.price} &euro;)
      <c:if test="${book.price < 30.0}">
         <img src='hot.png'
             alt='Moins de 30 &euro;'/>
      </c:if>
  </li>
</c:forEach>
</ul>
```

Utiliser la librairie `core` (bis)

```
<%
   String [] liste = Personne.listIds();
   pageContext.setAttribute ("liste", liste);
%>
<select name="personneId" size="<%= liste.length %>">
  <optgroup label="">
    <c:forEach items="${liste}" var="item" >
      <option value="${item}">
        <%= Personne.get((String)pageContext.getAttribute("item")).getNom()
  %>
      </option>
    </c:forEach>
  </optgroup>
</select>
```



Conditionnelle à choix multiple

```
<c:choose>
  <c:when test="${customer.category=='trial'}" >
    ...
  </c:when>
  <c:when test="${customer.category=='member'}" >
    ...
  </c:when>
    <c:when test="${customer.category=='preferred'}"
  >
    ...
  </c:when>
  <c:otherwise>
    ...
  </c:otherwise>
</c:choose>
```

Éléments actifs « xml »

Noyau	out, parse, set
Gestion du contrôle	choose (when, otherwise), forEach , if
Transformation	transform (param),

Evaluations xpath

```
<x:forEach var="book"
select="$applicationScope:booklist/books/*">
…
</x:forEach>
```

URL et documents xml

- Importer un document

```
<c:import url="/books.xml" var="xml" />
```

- Compiler le document

```
<x:parse doc="${xml}"
 var="booklist"   scope="application" />
```

i18n

Définir de la locale	`setLocale` `requestEncoding`
Gestion des messages	`Bundle, message` `param, setBundle`

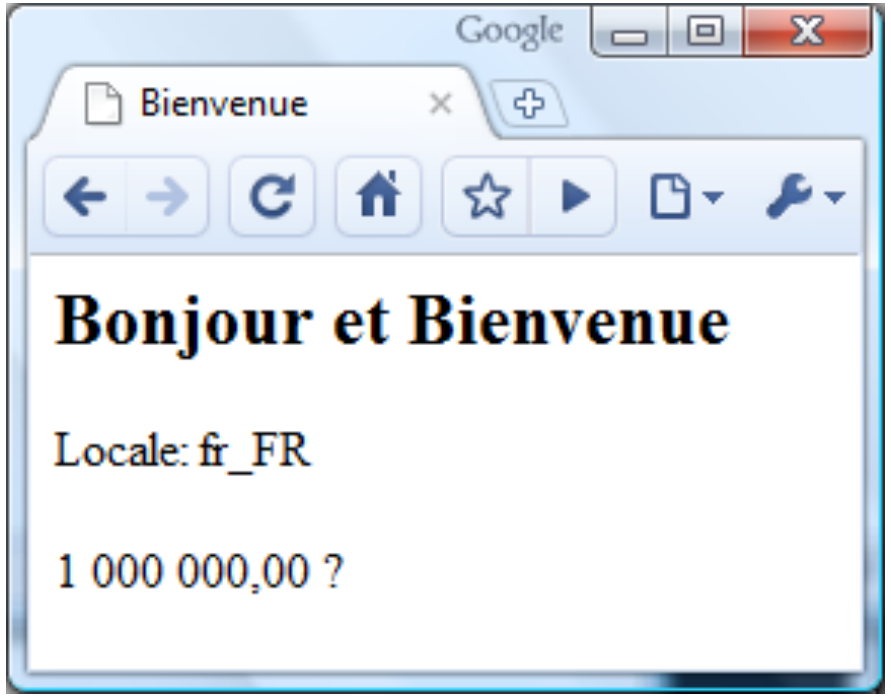
Nombres et dates	`formatNumber, formatDate`
	`parseDate, parseNumber`
	`setTimeZone, timeZone`

```
<h3><fmt:message key="Choose"/></h3>
```

Exemple1

```
<%@ taglib uri="http://java.sun.com/jstl/fmt"
  prefix="fmt" %>
<%@ page isELIgnored="false" %>
<html><fmt:setBundle basename="message"/>
<head><title><fmt:message key="Welcome" />
</title></head><body>
<h2><fmt:message key="Hello" /> <fmt:message key="and"
  /> <fmt:message key="Welcome" /></h2>
Locale:
  ${pageContext.request.locale.language}_${pageContext
  .request.locale.country}
<br /><br />
<fmt:formatNumber value="1000000" type="currency" />
</body></html>
```

Résultat


Google
Bienvenue
Bonjour et Bienvenue
Locale: fr_FR
1 000 000,00 ?

Properties file

- `message._fr.properties` placé à la racine des sources

```
Welcome=Bienvenue
Hello=Bonjour
and=et
```

Exemple i18n

```
<jsp:useBean id="now"
 class="java.util.Date" />
<jsp:setProperty name="now"
 property="time"   value="${now.tim
 e + 432000000}" />
<fmt:message key="ShipDate"/>
<fmt:formatDate value="${now}"
 type="date"   dateStyle="full"/>.
```

EL : Expression Language

Manipuler des donnée plus simplement qu'en Java

Utilisation dans les attributs de la librairie standard JSTL

```
<c:out value="${expression}" />
```

Où utiliser les EL

- Dans les attributs des tags JSP.
- Dans du texte simple de la page JSP.
  - Exemple

```
${monBean}
<prefix:actionTag id="newBean" param="${monBean}">
   ${newBean}
</prefix:actionTag>
```

Spécification

- Valable pour
  - **J2EE 1.4** (Servlet 2.4 / JSP 2.0)
  - **J2EE 1.3**. (**JSTL 1.0** -Java Standard Tag Library- avec un serveur d'application **J2EE 1.3)**
- La syntaxe est

`${ <expression> }`

- Exemples

```
${personne.age + 4}
${tab[index]% 4}
```

Quelques exemples

- **`${ pageContext.response.contentType }`**
  - affiche le content type de la réponse.
- **`${ pageScope["name"] }`**
  - affiche l'attribut "**name**" du scope page.
- **`${ param["page"] }`**
  - affiche la valeur du paramètre "**page**".
- **`${ header["user-agent"] }`**
  - affiche l'header "**user-agent**" envoyé par le navigateur.

Les objets implicites

- **pageContext** : Accès à l'objet **PageContext** de la page JSP.
- **pageScope** : Map permettant d'accéder aux différents attributs du scope **'page'**.
- **requestScope** : Map permettant d'accéder aux différents attributs du scope **'request'**.
- **sessionScope** : Map permettant d'accéder aux différents attributs du scope **'session'**.
- **applicationScope** : Map permettant d'accéder aux différents attributs du scope **'application'**.
- **param** : Map permettant d'accéder aux paramètres de la requête HTTP sous forme de **String**.
- **paramValues** : Map permettant d'accéder aux paramètres de la requête HTTP sous forme de **tableau de String**.
- **header** : Map permettant d'accéder aux valeurs du Header HTTP sous forme de **String**.
- **headerValues** : Map permettant d'accéder aux valeurs du Header HTTP sous forme de **tableau de String**.
- **cookie** : Map permettant d'accéder aux différents Cookies.
- **initParam** : Map permettant d'accéder aux **init-param**s du web.xml.

Recherche automatique d’un attribut

- `${name}`
  - Rechercher l’attribut "`name`" successivement dans les différentes portées (dans l'ordre : **page**, **request**, **session**, **application**).

Opérateurs

- Arithmétiques `+, -, *, /` (ou `div`) , `%` (ou `mod`)
- Relationnels `==` (ou `eq`) , `!=` (ou `ne`), `<` (ou `lt`), `>` (ou `gt`) , `<=` (ou `le`), `>=` (ou `ge`)
- Logiques `&&` (ou `and`) , `||` (ou `or`), `!` (ou `not[..]`)

Autres opérateurs

- `Empty[*]`

  true si l'opérande est **null**, une chaîne vide, un tableau vide, une Map vide ou une List vide. **false** sinon.

- `(c?v1:v2)`

  conditionnelle

Ecritures équivalentes

- `${ bean.name }`
- `${ bean["name"] }`
- `${ bean['name'] }`
- La valeur entre 'crochet' est une chaîne mais aussi n'importe quel objet.
- La méthode **toString()** est utilisé pour forcer le résultat à être une chaîne.
- `${ bean[ config.propertyName ] }`

Exemple d’écritures équivalentes

- `${ person.address.city }`
- `${ person.['address'].['city'] }`
- `${ person.["address"].["city"] }`
- `${ person.address.['city'] }`
- `${ person.["address"].city }`

Listes, tableaux et map

- Listes et tableaux
  - `${ list[0] }`
  - `${ list[1] }`
  - `${ list["2"] }`
  - `${ list["3"] }`
  - `${ list[ config.value ] }`
- Map
  - `${ map["clef1"] }`
  - `${ map['clef2'] }`
  - `${ map[ config.key ] }`

Prise en compte des expressions

- Dans une directive **page** ou dans un **tag** .
  - Utiliser l'attribut **isELIgnored**

```
<%@ page isELIgnored="false" %>
```

  Attention elles sont ignorées par défaut …

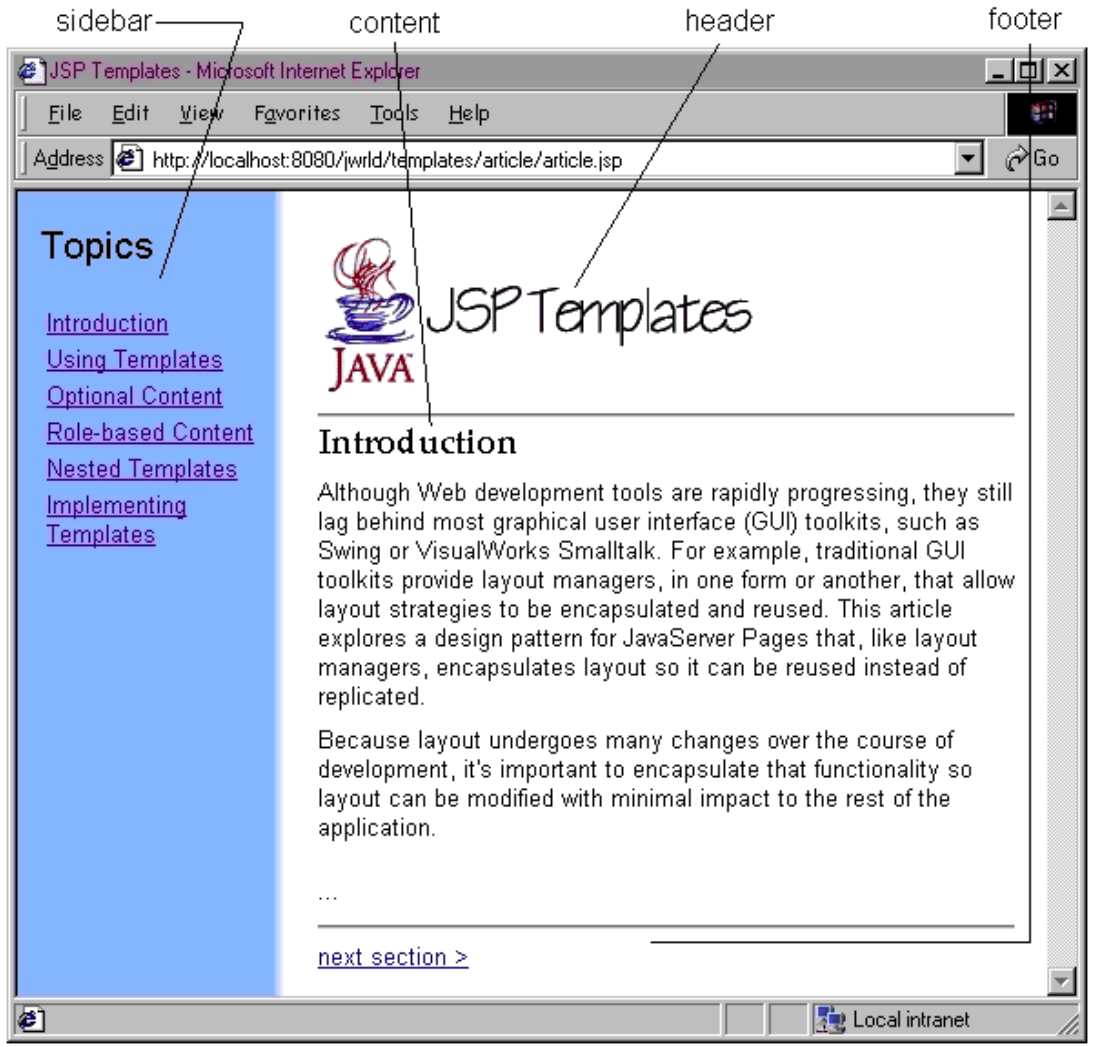
- Utiliser anti-slash pour signifier que ceci n'est pas une expression:

```
\${ ceci n'est pas une EL }
```

Un exemple de librairie de tags dédiée

« Modèlisation de pages WEB»

Un exemple de modèle de page



Solution 1

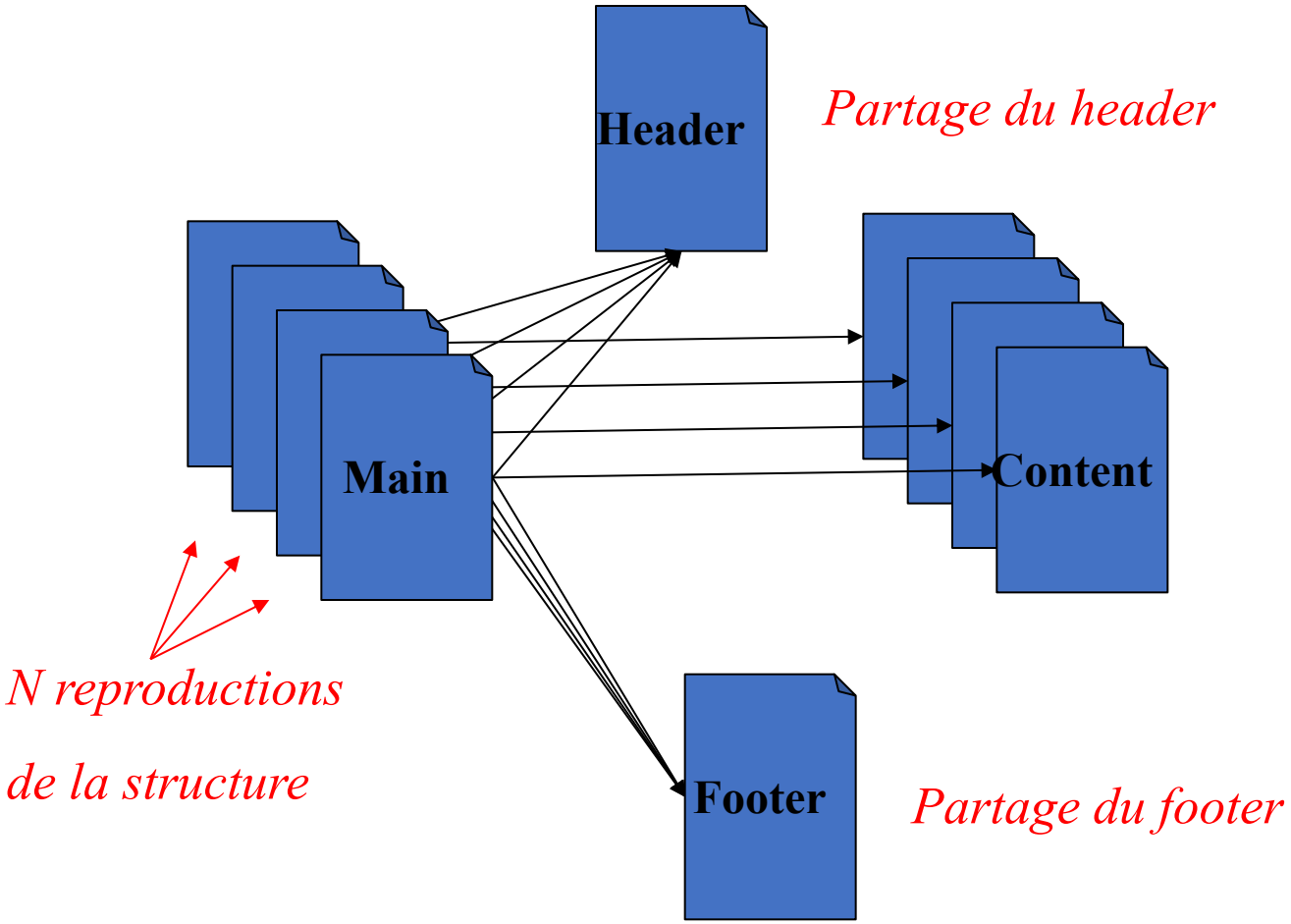
Utiliser la modulatité

Action `jsp:include`

Un modèle de page en JSP

```
<html><head><title>JSP Templates</title></head>
 <body background='graphics/background.jpg'>
  <table>
   <tr valign='top'>
    <td><%@include file='sidebar.html'%></td>
    <td><table>
      <tr><td><%@include file='header.html'%></td></tr>
      <tr><td><%@include file='intro.html'%></td></tr>
      <tr><td><%@include file='footer.html'%></td></tr>
     </table></td>
   </tr>
  </table>
 </body>
</html>
```

Structure du site WEB



Problèmes avec cette implémentation

- Le modèle de page est codé en dur dans la page elle même
  - Si plusieurs pages d'un site WEB ont le même modèle, celui-ci est dupliqué dans chaque page
  - Si on désire modifier la présentation il faut intervenir dans toutes les pages
- Nécessité d'un mécanisme pour inclure séparément le modèle de page et le contenu de la page

Solution 2

Utiliser un modèle de page unique

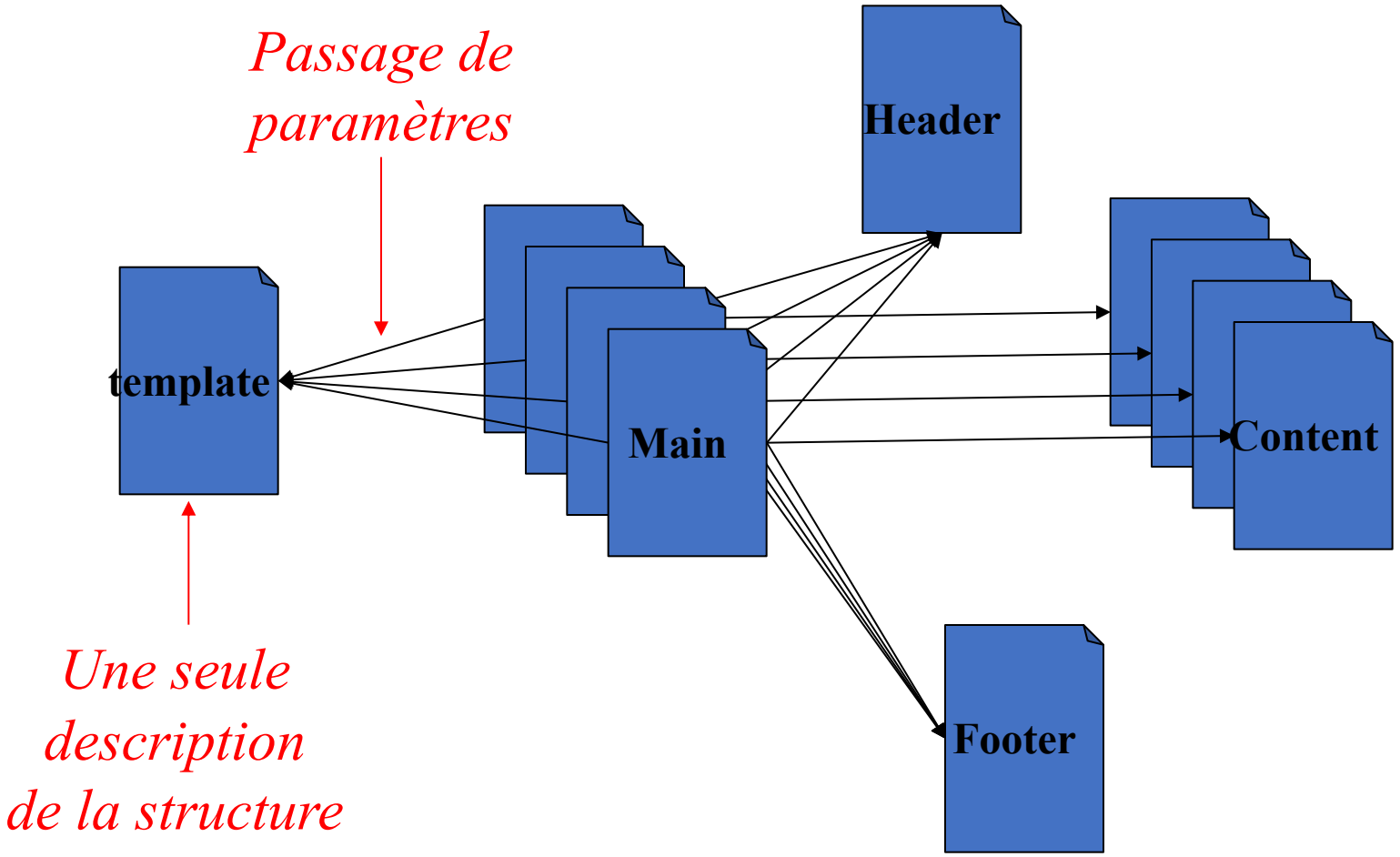
La libairie de tag *Template*

Un modèle de page générique

```
<%@ taglib uri='template.tld' prefix='template' %>
<html>
 <head><title><template:get name='title'/>
                                    </title></head>
 <body background='graphics/background.jpg'>
  <table>
   <tr valign='top'>
    <td><template:get name='sidebar'/></td>
    <td><table>
      <tr><td><template:get name='header'/></td></tr>
      <tr><td><template:get name='content'/></td></tr>
      <tr><td><template:get name='footer'/></td></tr>
     </table></td>
   </tr>
  </table>
 </body>
</html>
```

Déclaration de procédure
(5 paramètres)

Structurer un site WEB



Page utilisant un modèle

```
<%@ taglib uri='/WEB-INF/tlds/template.tld' prefix='template' %>

<template:insert template='/articleTemplate.jsp'>
  <template:put name='title' content='Templates'
    direct='true'/>
  <template:put name='header' content='/header.html' />
  <template:put name='sidebar' content='/sidebar.jsp' />
  <template:put name='content'
    content='/introduction.html'/>
  <template:put name='footer' content='/footer.html' />
</template:insert>
```

Appel de procédure

articleTemplate.jsp(
Templates, /header.html,
/sidebar.jsp, /content.html,
/footer.html
)

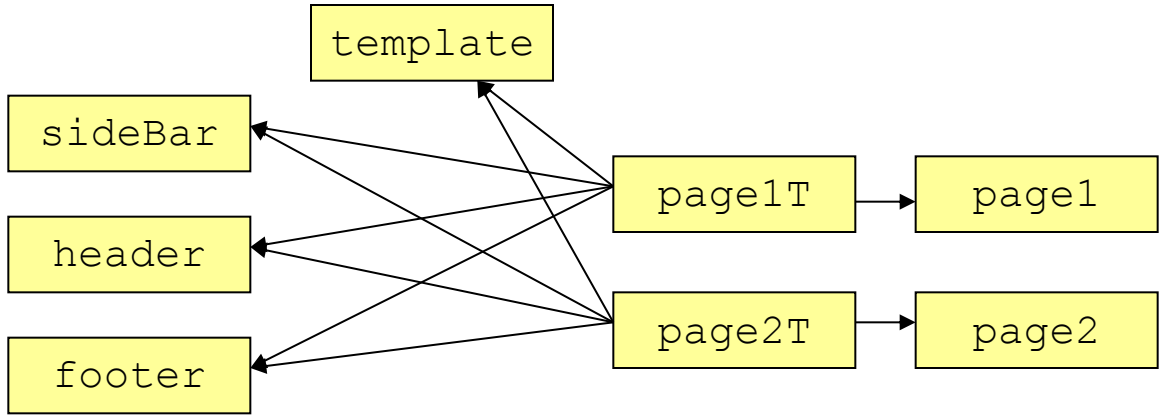
Fonctionnement

- Appel de procédure

```
<template:insert
  template='/articleTemplate.jsp'>
```

- Passage de paramètre effectif

```
<template:put name='footer'
  content='/footer.html'/>
```



Attribut direct

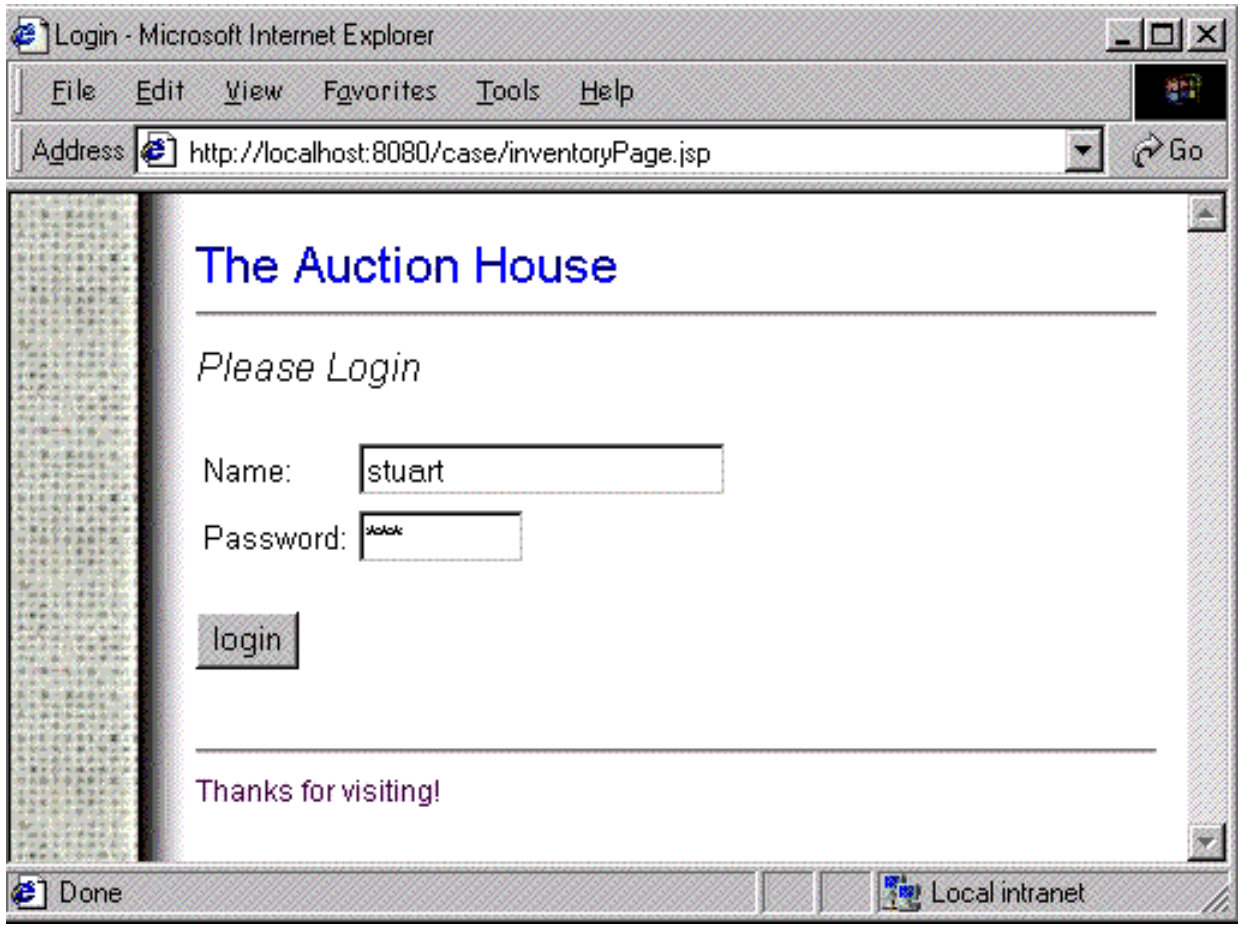
```
<template:put name='title' content='Templates' direct='true'/>
```

Le valeur de l'attribut `content` est directement inséré

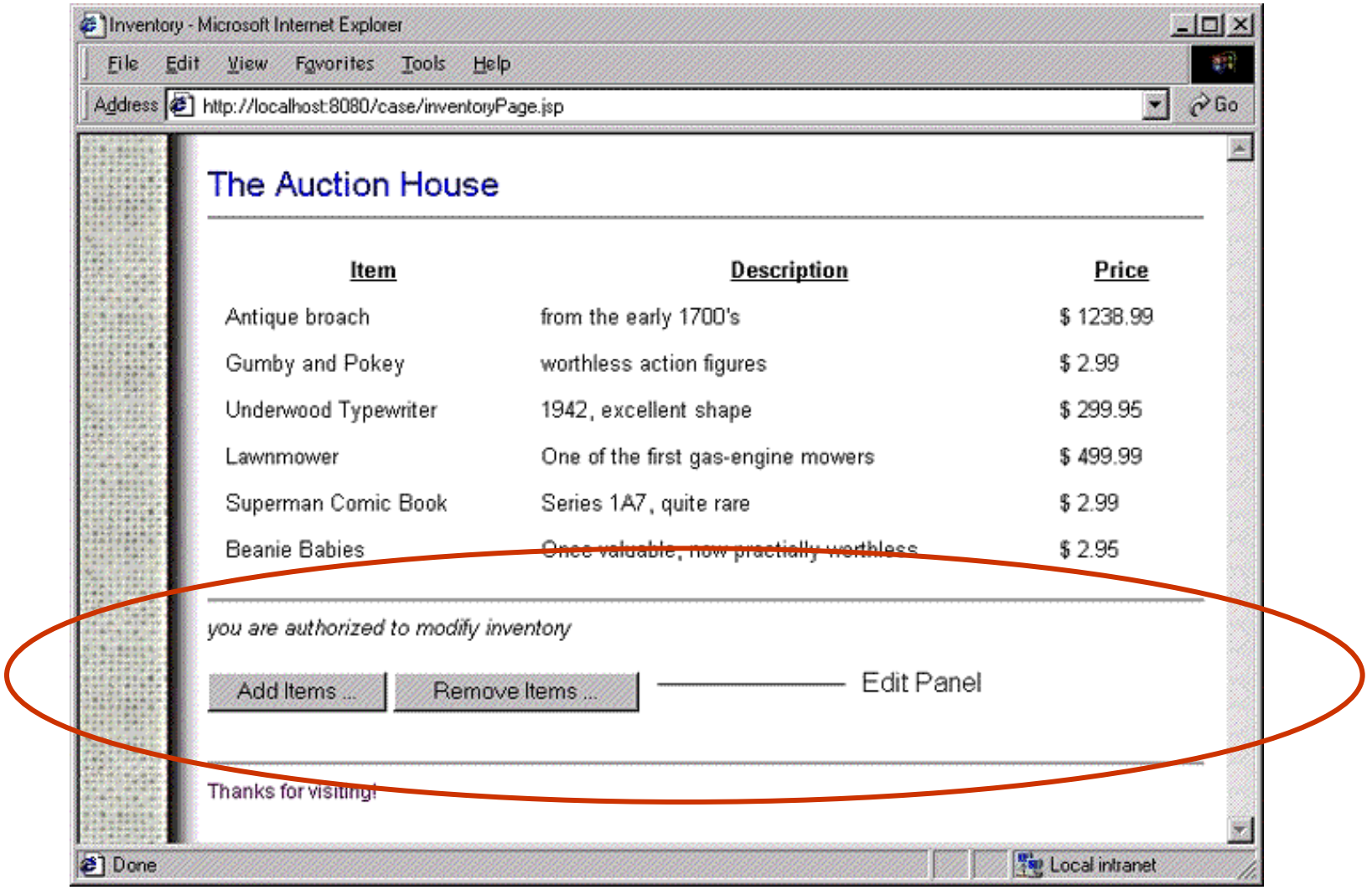
Contenu optionnel

- Tout contenu d'un modèle est optionnel
  - Un même modèle peut donc servir pour plusieurs format …

Modèle à trois zones



Modèle à quatre zones



Conclusion sur les templates JSP

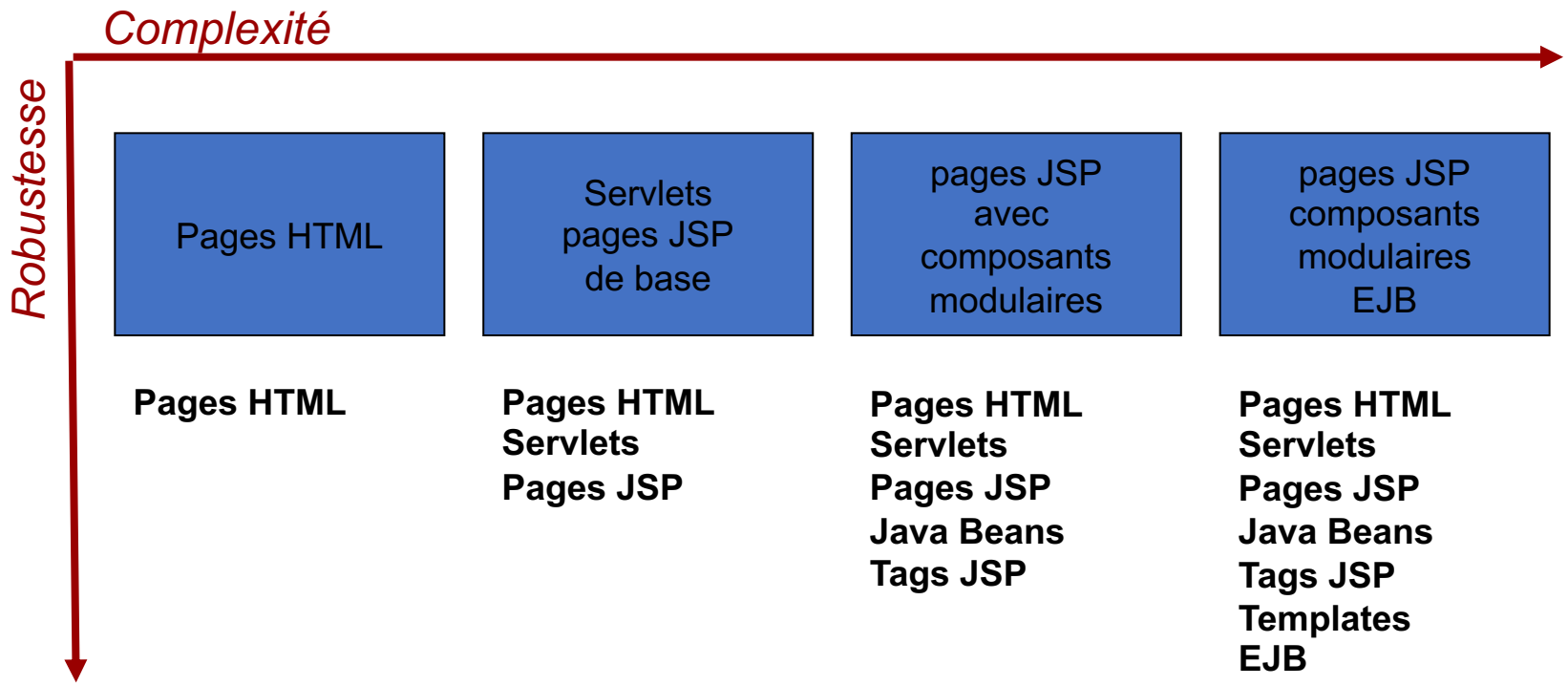
- Avantages
  - Paramétrage de la présentation d'un site web par l'utilisation de modèles de page
- Idées tirées de
  - *Java Tip 98: Reflect on the Visitor design pattern*

  Implement visitors in Java, using reflection, By David Geary
  - `http://www.javaworld.com`
- Où trouver la librairie
  - `http://www.javaworld.com/javaworld/jw-09-2000/jspweb/jw-0915-jspweb.zip`

Conclusion servlet/jsp ouverture sur le monde java

- Communiquer avec
  - des classes
  - des applets
- Librairies
  - jdbc
  - xml (dom, sax, transformation sxl, base de données native xml xindice)
  - soap, xmlrpc ...

Conception d'applications WEB



Références

- Rappel : Tutorial de sun (1500 pages)
  - (pdf) http://java.sun.com/j2ee/1.4/docs/tutorial-update2/doc/J2EETutorial.pdf
- Un tutorial en Français sur les tags
  - http://adiguba.developpez.com/tutoriels/j2ee/jsp/taglib/
- Un tutorial sur le développement J2EE avec Eclipse et Tomcat
  - ftp://ftp2.developpez.biz/developpo/users/tahe/fichiers/progwebjavaavececlipseettomcat.pdf